Design and Implementation of a Custom Multithreaded DNS Server with TTL-Based Caching and Persistent MongoDB Search History

Narne Adithya Hanuman G Sreemanth V. Umika
Slot: F1 | Team: Team 13
Guided by Dr. Anitha E

***Abstract*—The Domain Name System (DNS) translates human-readable domain names into IP addresses and is a core service of every network application. This paper presents the design and implementation of a custom DNS resolution system written in Java. The system comprises a multithreaded UDP server, an interactive client, and a Swing-based graphical console. Requests are exchanged using a lightweight pipe-delimited text protocol. Each query is resolved through a three-stage lookup pipeline: an in-memory cache with per-entry time-to-live (TTL), a persistent MongoDB search history, and finally a local records file with fallback to the operating system's upstream resolver. Every query is also logged to a separate MongoDB collection, and a utility imports hostnames from Google Chrome's browsing history. The server degrades gracefully to in-memory operation when MongoDB is unavailable. A scripted benchmark exercises cold lookups, warm cache hits, upstream resolution, negative lookups and mixed traffic.**

***Keywords*—DNS, UDP sockets, multithreading, caching, TTL, MongoDB, Java, computer networks**

I. Introduction

Every interaction on the Internet begins with a name lookup. The Domain Name System (DNS) maps hierarchical names such as example.com to numeric IP addresses and is described in RFC 1034 and RFC 1035 [1], [2]. Because a single page load can trigger many lookups, resolvers rely heavily on caching, and the effectiveness of that caching largely determines the latency users perceive [5].

Production resolvers are complex, so their internal behaviour is hard to observe in a teaching setting. This project builds a compact, fully observable DNS service from first principles using the Java socket API. It demonstrates several computer-networking concepts in one system: connectionless UDP communication [3], an application-layer protocol, concurrent request handling, TTL-driven cache expiry, and integration with a persistent database.

The main contributions are:

- a multithreaded UDP DNS server that handles each request in its own thread;
- a thread-safe in-memory cache with per-entry TTL, hit/miss/eviction statistics and a live hit-rate metric;
- a MongoDB-backed search history with atomic visit counters and a separate per-query audit log;
- a Chrome browsing-history importer, a command-line client and a three-tab graphical console; and
- a benchmark that demonstrates cache and database behaviour end to end.

II. Background

A stub resolver sends a query to a DNS server and waits for a single answer. Real DNS messages are binary and carry a record type, class and TTL as defined in RFC 1035 [2]. DNS normally runs over UDP because a query and its answer each fit in one datagram, avoiding the connection set-up cost of TCP [3], [6].

Caching is central to DNS scalability. Each resource record carries a TTL telling resolvers how long the answer may be reused. Entries that outlive their TTL must be discarded and re-fetched. Negative answers can also be cached [4]. This project reproduces the TTL mechanism at a stub-resolver level and layers a persistent history store on top of it.

III. System Architecture

A. Overview

The system follows a client–server model (Fig. 1). The `DNSServer` class binds a `DatagramSocket` to UDP port 5454 (moved from the conventional 5353, which is occupied by multicast DNS on many hosts), loads the records file into a `HashMap`, creates one shared `DNSCache`, and initialises the `MongoHistoryService` singleton. It then loops forever receiving datagrams.

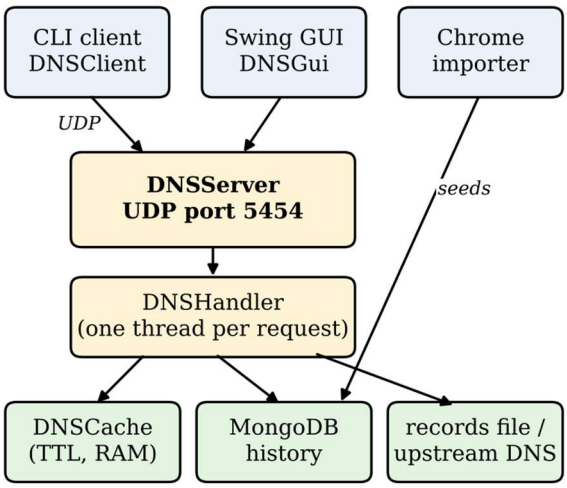


Fig. 1. High-level architecture of the custom DNS system.

B. Application-Layer Protocol

Clients and server exchange plain-text, pipe-delimited messages in a single UDP datagram of at most 1024 bytes (Table I). The response carries a source tag and a trailing value whose meaning depends on the source: remaining TTL for cache hits, the cumulative search count for MongoDB hits, and the default TTL for fresh lookups.

TABLE I
PROTOCOL MESSAGE FORMATS

Type	Format
Request	`DNS|REQUEST|<domain>`
Success	`DNS|RESPONSE|<domain>|<ip>|<source>|<value>`
Error	`DNS|ERROR|<domain>|<reason>`
Sources	`CACHE`, `MONGODB`, `DNS_LOOKUP`
Reasons	`DOMAIN_NOT_FOUND`, `INVALID_REQUEST_FORMAT`, `INVALID_DOMAIN_NAME`

C. Concurrency Model

For every received datagram the server starts a new daemon thread running a `DNSHandler`. Clients are therefore served concurrently and a slow upstream lookup for one client does not block the others. The receive loop never performs lookup work itself. Sends on the shared socket are wrapped in a synchronized block so that responses are not interleaved.

IV. IMPLEMENTATION

A. Technology Stack

The system is written in Java 17 and built with Maven. Dependencies are the MongoDB synchronous driver 4.11.1, SQLite JDBC 3.45.1 (for the Chrome importer) and SLF4J. The user interface uses Java Swing.

B. TTL-Based DNS Cache

`DNSCache` stores entries in a `LinkedHashMap`, each holding the IP address, the caching timestamp and a TTL (default 60 s). All public methods are synchronized, which makes the cache safe under concurrent handler threads. An entry is evicted lazily when a lookup finds it expired, and `evictExpired()` can purge all stale entries in bulk. The cache counts hits, misses and evictions, and exposes the hit rate

$$H = N_{hit} / (N_{hit} + N_{miss}) \times 100\% \quad (1)$$

where expired lookups count as misses. If h is the hit ratio, and Tcache and Tmiss are the latencies of a cache hit and a miss, the expected resolution time is

$$T_{avg} = h \cdot T_{cache} + (1 - h) \cdot T_{miss} \quad (2)$$

Since a cache hit is far cheaper than a database or network lookup, raising h directly lowers the average latency.

C. MongoDB Persistence

`MongoHistoryService` (a thread-safe singleton) connects using the `MONGODB_URI`, `MONGODB_DATABASE` and `MONGODB_COLLECTION` environment variables, so no credentials are hard-coded. Two collections are used in the `custom_dns` database:

- `search_history` keeps one document per domain with its IP, source, search count and last-searched time. A unique index on domain prevents duplicates, and a hit increments the count and timestamp atomically in one update operation.
- `dns_queries` stores every individual query as its own document and is indexed by descending timestamp for fast retrieval of recent activity.

The server selection timeout is 2.5 s. If the connection fails, the service marks itself offline and all history calls return empty results, so DNS resolution continues unaffected.

D. Local Records and Upstream Fallback

`dns_records.txt` holds `<domain>=<ip>` entries (19 in the supplied file) covering well-known public domains, private `.local` hosts and teaching examples. Comments and blank lines are ignored and malformed lines are reported and skipped. A domain missing from the file is passed to `InetAddress.getByName()`, so the server can also answer for real Internet hosts.

E. Chrome History Importer

`ChromeHistoryImporter` locates Chrome's SQLite History file, copies it to a temporary file to avoid the browser's file lock, and reads the `urls` table. It extracts unique hostnames of HTTP and HTTPS pages, aggregates visit counts, resolves each host (local records first, then real DNS) and upserts the result into `search_history` with source `CHROME_HISTORY`.

F. Client Interfaces

`DNSClient` is an interactive console client. It validates input, sends a request, applies a 5 s socket timeout, measures the round-trip time and prints a formatted result. It handles empty input, timeouts, unreachable servers, malformed responses and every error code in Table I. `DNSGui` is a Swing console with three tabs: Server (start/stop, live log, cache table with TTL countdown, hit/miss/entry/query counters and cache flush), Client (lookup field, result panel, MongoDB history table, reload and Chrome import buttons) and Query Log (recent entries from `dns_queries`).

V. Resolution Algorithm

A. Lookup Order

`DNSHandler` decodes the datagram, validates the `DNS|REQUEST|<domain>` format, lowercases the domain and rejects empty names. The domain is then resolved in the strict priority order shown in Fig. 2 and Table II.

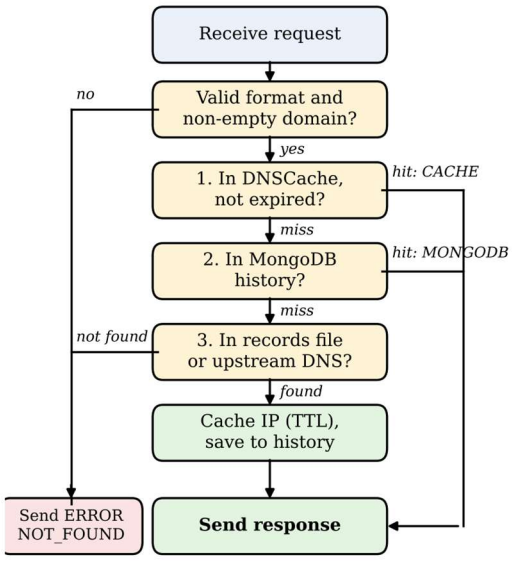


Fig. 2. Request resolution flow in DNSHandler.

TABLE II
Lookup stages and their effects

Stage	Source tag	Side effects
1. Cache	`CACHE`	None; returns remaining TTL
2. MongoDB	`MONGODB`	Increments search count; refills cache
3a. Records file	`DNS_LOOKUP`	Caches IP; saves new history entry
3b. Upstream DNS	`DNS_LOOKUP`	Caches IP; saves new history entry
Failure	`ERROR`	Returns `DOMAIN_NOT_FOUND`

Placing the in-memory cache first gives the fastest path, while the MongoDB history lets previously seen names be answered after a cache expiry or a server restart. Only if both miss does the server consult the records file or the upstream resolver. A successful result is cached with the default TTL and persisted, so the next request for the same name is served from memory.

B. Pseudocode

Algorithm 1 gives the server loop and Algorithm 2 the per-request lookup performed by each handler thread. Algorithm 3 shows the TTL cache operations that Algorithm 2 relies on.

Algorithm 1. Server loop (DNSServer)

```
records = LOAD(dns_records.txt)
cache   = NEW DNSCache(ttl = 60)
mongo   = MongoHistoryService.GET()
socket  = NEW UDPSocket(port = 5454)
FOREVER DO
  packet = socket.RECEIVE()
  SPAWN thread DNSHANDLER(packet)
// the loop never performs lookup work
```

ALGORITHM 2. HANDLER (DNSHANDLER.RUN)

```
FUNCTION HANDLE(packet)
  msg = DECODE(packet)
  parts = SPLIT(msg, '|')
  IF parts != [DNS, REQUEST, d] THEN
    reply = ERROR(INVALID_REQUEST_FORMAT)
  ELSE
    reply = RESOLVE(lower(TRIM(d)))
  SYNCHRONIZED(socket) socket.SEND(reply)

FUNCTION RESOLVE(d)
  IF d is empty THEN
    RETURN ERROR(INVALID_DOMAIN_NAME)
  e = cache.GET(d)                // stage 1
  IF e != null THEN
    RETURN RESPONSE(d, e.ip, CACHE, e.ttlLeft)
  r = mongo.FIND_AND_RECORD(d)    // stage 2
  IF r != null THEN
    cache.PUT(d, r.ip)
    RETURN RESPONSE(d, r.ip, MONGODB, r.count)
  ip = records[d]                 // stage 3a
  IF ip == null THEN              // stage 3b
    TRY ip = OS_LOOKUP(d)
    CATCH UnknownHost
      RETURN ERROR(DOMAIN_NOT_FOUND)
  cache.PUT(d, ip)
  mongo.SAVE_NEW(d, ip, DNS_LOOKUP)
  RETURN RESPONSE(d, ip, DNS_LOOKUP, cache.ttl)
```

ALGORITHM 3. TTL CACHE (DNSCACHE)

```
FUNCTION GET(d)                   // synchronized
  e = map[lower(d)]
  IF e == null THEN
    misses++ ; RETURN null
  IF now >= e.cachedAt + e.ttl THEN
    REMOVE map[lower(d)]
    evictions++ ; misses++
    RETURN null                   // expired = miss
  hits++
  RETURN e

FUNCTION PUT(d, ip, ttl = default)
  map[lower(d)] = Entry(ip, now, ttl)

FUNCTION HIT_RATE()
  RETURN hits / (hits + misses) * 100
```

Algorithm 4 shows the persistent history operations, including the offline path, and Algorithm 5 the Chrome importer that seeds the history.

ALGORITHM 4. MONGODB HISTORY WITH OFFLINE MODE

```
FUNCTION FIND_AND_RECORD(d)
  IF NOT online THEN RETURN null
  TRY
    // one atomic update
    doc = FIND_ONE_AND_UPDATE(
            {domain: d},
            INC searchCount,
            SET lastSearched = now)
    RETURN doc
  CATCH MongoError
    online = false              // degrade
    gracefully
    RETURN null

FUNCTION SAVE_NEW(d, ip, src)
  IF online THEN
    UPSERT search_history {d, ip, src,
                           count = 1, now}
    INSERT dns_queries {d, ip, src, now}
```

ALGORITHM 5. CHROME HISTORY IMPORT

```
FUNCTION IMPORT_HISTORY()
  f   = LOCATE(Chrome History file)
  tmp = COPY(f)       // avoid browser file lock
  rows = SQL(tmp, 'SELECT url, visit_count
                   FROM urls')
  FOR EACH row IN rows DO
    IF scheme(row.url) IN {http, https} THEN
      host = HOSTNAME(row.url)
      visits[host] += row.visit_count
  FOR EACH host IN visits DO
    ip = records[host]
    IF ip == null THEN ip = OS_LOOKUP(host)
    mongo.UPSERT(host, ip, visits[host],
                 CHROME_HISTORY)
```

VI. TESTING AND EVALUATION

Two programs are provided for verification. `MongoDnsTest` checks the lookup priority: a first query for google.com must miss the cache and MongoDB, be resolved and stored with a count of one; after the cache is flushed, a second query must be answered from MongoDB with an incremented count.

`CacheStressTest` first probes whether a server is listening on port 5454. If so, it runs the four live network scenarios in Table III and reports per-query latency and the source tag. If not, it falls back to a direct in-memory simulation of the cache engine with TTL decay.

TABLE III
BENCHMARK SCENARIOS

No.	Scenario	Expected behaviour
1	Cold vs. hot lookup of a local-record domain	First miss, then `CACHE` hit
2	Domain absent from the file (real upstream DNS)	Upstream resolve, then cached
3	Non-existent domain	`DOMAIN_NOT_FOUND` error
4	Mixed workload of 20 queries	Repeats served from cache

The expected outcome is that a cold query pays the cost of database and possibly network resolution, whereas a repeat query within the TTL window is answered from memory with the remaining TTL reported in the response. Exact latency figures depend on the host and on whether MongoDB is local or hosted, so they should be measured on the deployment machine.

VII. Limitations and Future Work

- The protocol is a custom text format and is not interoperable with standard DNS clients such as nslookup or dig. Implementing the RFC 1035 binary wire format would remove this limitation.
- Only a single address per name is supported, and record types (AAAA, MX, CNAME) are not modelled.
- A new thread is created per request without an upper bound. A fixed thread pool would resist floods better.
- The cache has no maximum size, and negative answers are not cached [4].
- Documents in `search_history` have no expiry, so an IP stored in MongoDB can become stale. A TTL index or periodic re-validation would address this.
- UDP requests are not authenticated, so spoofing and amplification are possible in an open deployment.

VIII. Conclusion

This paper described a custom DNS server and client implemented in Java. The server combines a thread-per-request UDP design, a TTL-aware in-memory cache and a persistent MongoDB search history behind a clear three-stage lookup pipeline. The GUI, query log, Chrome importer and benchmark make the behaviour of each layer directly observable. The project illustrates how caching, concurrency and persistence interact in a name-resolution service and offers a base for extending towards a standards-compliant resolver.

Acknowledgment

The authors thank the faculty of the Computer Networks course for their guidance.

References

[1] P. Mockapetris, “Domain names—Concepts and facilities,” IETF, RFC 1034, Nov. 1987.

[2] P. Mockapetris, “Domain names—Implementation and specification,” IETF, RFC 1035, Nov. 1987.

[3] J. Postel, “User Datagram Protocol,” IETF, RFC 768, Aug. 1980.

[4] M. Andrews, “Negative caching of DNS queries (DNS NCACHE),” IETF, RFC 2308, Mar. 1998.

[5] J. Jung, E. Sit, H. Balakrishnan, and R. Morris, “DNS performance and the effectiveness of caching,” IEEE/ACM Trans. Netw., vol. 10, no. 5, pp. 589–603, Oct. 2002.

[6] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 7th ed. Boston, MA, USA: Pearson, 2017.